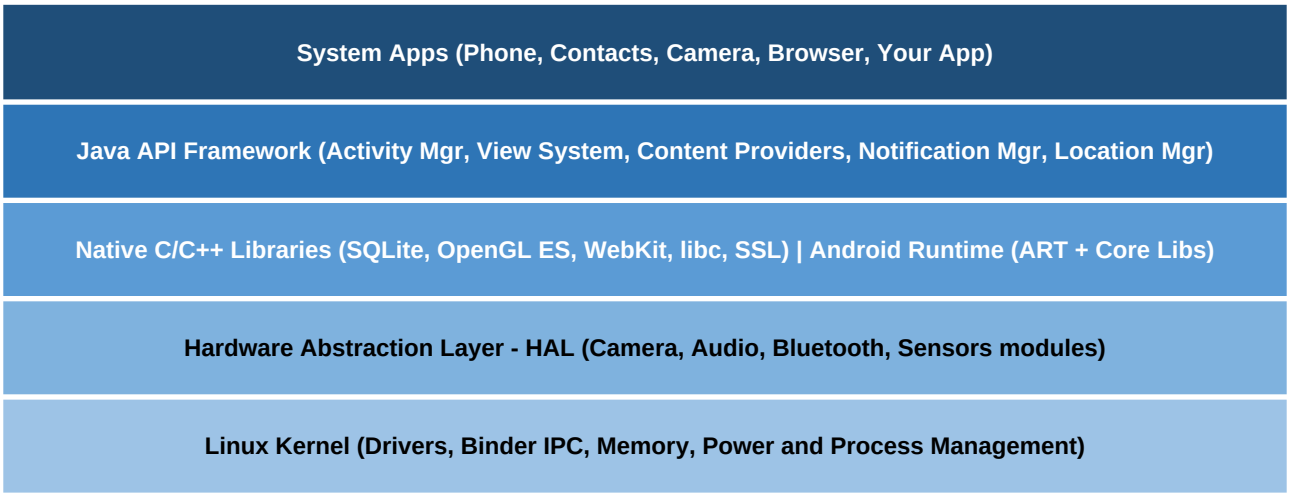
Mobile Application Development (3170726)

Viva Answers, Set 2: Architecture, Lifecycles, Menus, Firebase, JSON and Web Services

1. Android Architecture, Activity Lifecycle and Fragment Lifecycle

1.1 Android Architecture

Android is an open-source, Linux-based software stack organised into layers. Each layer provides services to the layer above it.



Layer	Role
Linux Kernel	Bottom layer. Provides device drivers (display, camera, Wi-Fi, Bluetooth), memory, process and power management, and Binder IPC.
HAL	Standard interfaces that expose hardware capabilities to the higher-level framework, so the framework does not depend on specific drivers.
Native Libraries	C/C++ libraries such as SQLite (database), OpenGL ES (graphics), WebKit (browser engine), libc, SSL and Media Framework.
Android Runtime (ART)	Runs the app's DEX bytecode. Uses Ahead-Of-Time (AOT) compilation for faster launch and better battery. Replaced Dalvik (JIT).
Java API Framework	APIs developers use directly: Activity Manager, View System, Content Providers, Resource Manager, Notification Manager, Location Manager, Package Manager, Telephony Manager.
System Apps	Top layer: built-in apps (Phone, Contacts, Browser) and third-party apps, including yours.

1.2 Activity Lifecycle

An Activity is a single screen with a UI. It moves through states, and Android calls a callback method at every transition.

Method	When it is called	Typical use
onCreate()	Activity is first created	Set the layout (setContentView), initialise views, restore saved state
onStart()	Activity becomes visible	Prepare things the user will see
onResume()	Activity is in foreground and interactive	Start animations, camera, sensors
onPause()	Another activity comes in front (partially visible)	Pause animations, save light data, release sensors
onStop()	Activity is no longer visible	Release heavy resources, save data

Method	When it is called	Typical use
onRestart()	Returning from the stopped state	Re-prepare before onStart()
onDestroy()	Activity is finishing or being destroyed by the system	Final cleanup

Flow: onCreate() > onStart() > onResume() > (Running) > onPause() > onStop() > onDestroy(). From onStop() the user may come back through onRestart() > onStart() > onResume(). From onPause() the user may return directly to onResume().

Three nested loops: Entire lifetime = onCreate to onDestroy. Visible lifetime = onStart to onStop. Foreground lifetime = onResume to onPause.

Screen rotation: the activity is destroyed and recreated. Save data with onSaveInstanceState() or use a ViewModel.

1.3 Fragment Lifecycle

A Fragment is a reusable part of the UI with its own lifecycle, always hosted inside an Activity. Its lifecycle is tied to the host activity: if the activity is paused, all its fragments are paused.

Method	Purpose
onAttach()	Fragment is attached to its host activity (context is available)
onCreate()	Fragment is created; initialise non-UI components
onCreateView()	Inflate and return the fragment's layout
onViewCreated()	View is ready; find views and set listeners
onStart() / onResume()	Fragment becomes visible, then interactive
onPause() / onStop()	Fragment loses focus, then is no longer visible
onDestroyView()	Fragment's view is being removed; clean up view references
onDestroy()	Fragment is destroyed
onDetach()	Fragment is detached from the activity

Flow: onAttach > onCreate > onCreateView > onViewCreated > onStart > onResume > onPause > onStop > onDestroyView > onDestroy > onDetach.

Key difference: Fragments have extra callbacks (onAttach, onCreateView, onDestroyView, onDetach) because they must attach to an activity and build or destroy their own view separately.

2. How to Convert a Java Class into an Android Class

A plain Java class becomes an Android component when it **extends an Android framework class**, is **connected to a layout** and is **registered in the manifest**.

Steps

1. Extend an Android base class: **AppCompatActivity** (or Activity) for a screen, Fragment, Service, BroadcastReceiver or ContentProvider.
2. Override the lifecycle method **onCreate(Bundle savedInstanceState)** and call **super.onCreate()**.
3. Attach the UI with **setContentView(R.layout.activity_main)**.
4. Get views with **findViewById(R.id.xxx)** and add listeners.
5. Register the class in **AndroidManifest.xml** using the <activity> tag (otherwise the app crashes on launch).
6. Add the required imports from the android.* and androidx.* packages.

Plain Java class

```
public class MainScreen {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Converted Android class

```
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {   // 1. extend
    @Override
    protected void onCreate(Bundle savedInstanceState) {  // 2. lifecycle
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);           // 3. layout

        TextView tv = findViewById(R.id.textView1);       // 4. view
        tv.setText("Hello");
    }
}
```

Manifest entry

```
<activity android:name=".MainActivity" />
```

Note: A normal helper class (for example a Student model or a utility class) does not need to extend anything. It works as plain Java. It needs a **Context** object only when it must use Android services such as SharedPreferences, Toast or resources.

3. In Which File Is the UI Layout Code Written?

The UI layout is written in an **XML file** inside the **res/layout** folder, for example **activity_main.xml**. The Java or Kotlin code only loads it using setContentView(R.layout.activity_main).

```
<!-- res/layout/activity_main.xml -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="Hello Android" />
</LinearLayout>
```

- **Why XML:** separates design (XML) from logic (Java), and gives a different layout per screen size or orientation using folders like layout-land and layout-sw600dp.
- **Alternatives:** the UI can also be created programmatically in Java/Kotlin code, or with Jetpack Compose (Kotlin functions with no XML).

4. When You Run an Android Studio Project, Which File Runs First?

The system first reads **AndroidManifest.xml** to find the **launcher activity**, the one with the MAIN action and LAUNCHER category. That activity's Java class (usually **MainActivity.java**) starts, and its **onCreate()** runs first. onCreate() then loads the layout **activity_main.xml**.

```
<activity android:name=".MainActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Execution order: AndroidManifest.xml > MainActivity.java (onCreate) > activity_main.xml (via setContentView) > R.java IDs resolve the resources.

Extra point: If the app declares a custom Application class (android:name in the <application> tag), its onCreate() runs even before any activity. The build itself is driven by build.gradle.

5. Menu and Its Types

A menu is a list of actions or options shown to the user. It is defined as an XML resource in **res/menu/** and inflated in code. Menus give users actions without cluttering the screen.

Type	Description	How it is shown / key methods
Options Menu	Primary menu for the activity (Settings, Search, Share). Appears in the app bar or the three-dot overflow.	onCreateOptionsMenu() to inflate; onOptionsItemSelected() to handle clicks
Context Menu	Floating menu on long-press of a view. Also available as a contextual action mode (action bar for multi-select).	registerForContextMenu(view); onCreateContextMenu(); onContextItemSelected()
Popup Menu	Modal list anchored to a specific view, shown on click.	PopupMenu pm = new PopupMenu(ctx, view); pm.inflate(); pm.show(); setOnMenuItemClickListener()

Menu XML (res/menu/main_menu.xml)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
    <item android:id="@+id/action_settings"
          android:title="Settings"
          app:showAsAction="never" />
    <item android:id="@+id/action_search"
          android:title="Search"
          android:icon="@drawable/ic_search"
          app:showAsAction="ifRoom" />
</menu>
```

Options menu code

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main_menu, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
    if (item.getItemId() == R.id.action_settings) {
        Toast.makeText(this, "Settings", Toast.LENGTH_SHORT).show();
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

Popup menu code

```
PopupMenu popup = new PopupMenu(this, button);
popup.getMenuInflater().inflate(R.menu.main_menu, popup.getMenu());
popup.setOnMenuItemClickListener(item -> {
    // handle click using item.getItemId()
    return true;
});
popup.show();
```

Related navigation menus: Navigation Drawer (side menu) and Bottom Navigation also use menu XML files.

6. Firebase, JSON and Web Service

6.1 Firebase

Firebase is Google's **Backend-as-a-Service (BaaS)** platform. It gives apps a ready-made backend (database, login, storage, notifications) without writing or maintaining server code.

Service	Purpose
Authentication	Login with email/password, Google, phone OTP and others
Realtime Database	Cloud-hosted NoSQL database stored as JSON tree, syncs data in real time
Cloud Firestore	Newer NoSQL document database with richer queries and scaling
Cloud Storage	Store and serve images, audio, video and files
Cloud Messaging (FCM)	Send push notifications
Analytics / Crashlytics	Usage tracking and crash reporting
Hosting / Remote Config	Web hosting; change app behaviour without an update

Setup steps: create a project in the Firebase console > register the Android app (package name) > download google-services.json into the app folder > add the Google services plugin and Firebase dependencies in Gradle > sync.

```
// Write to Realtime Database
DatabaseReference ref = FirebaseDatabase.getInstance().getReference("users");
ref.child("101").setValue(new User("Rahul", "rahul@mail.com"));

// Login with Firebase Authentication
FirebaseAuth.getInstance()
    .signInWithEmailAndPassword(email, password)
    .addOnCompleteListener(task -> {
        if (task.isSuccessful()) { /* logged in */ }
    });
```

Advantages: no server to manage, real-time sync, free tier, easy SDK, scalable. **Disadvantage:** vendor lock-in and cost at large scale.

6.2 JSON (JavaScript Object Notation)

JSON is a lightweight, text-based, language-independent format for exchanging data, mostly between a server and an app. It is easier to read and smaller than XML.

- Data is stored as **key-value pairs** inside { } for objects and [ ] for arrays.
- Keys are strings in double quotes. Values can be string, number, boolean, null, object or array.

```
{
  "name": "Rahul",
  "age": 21,
  "isStudent": true,
  "subjects": ["MAD", "CN", "AI"]
}
```

Parsing in Android with the built-in org.json classes

```
String json = "{\"name\":\"Rahul\",\"age\":21}";
try {
    JSONObject obj = new JSONObject(json);
    String name = obj.getString("name");
    int age = obj.getInt("age");
} catch (JSONException e) {
    e.printStackTrace();
}
// JSONArray arr = new JSONArray(text); arr.getJSONObject(0);
```

Classes: JSONObject (for { }), JSONArray (for [ ]), JSONException (parse error). Libraries such as Gson or Moshi convert JSON directly to Java objects.

6.3 Web Service

A web service is software hosted on a server that lets different applications communicate over a network (usually HTTP) using standard data formats. A mobile app calls the web service to fetch or send data, for example login, product lists or weather.

Point	SOAP	REST
Full form	Simple Object Access Protocol	Representational State Transfer
Nature	Protocol with strict rules	Architectural style
Data format	XML only	JSON, XML, text
Weight	Heavy	Lightweight, fast
Use in mobile	Rare	Most common

HTTP methods in REST: GET (read), POST (create), PUT (update), DELETE (remove).

Android flow: App sends HTTP request > server processes it > server returns JSON > app parses JSON > UI is updated. Networking must run off the main thread, and the app needs the INTERNET permission.

```
<uses-permission android:name="android.permission.INTERNET" />
```

Libraries: Retrofit (type-safe, annotation-based), Volley, OkHttp, HttpURLConnection (basic).

```
// Retrofit example
public interface ApiService {
    @GET("users/{id}")
    Call<User> getUser(@Path("id") int id);
}

Retrofit retrofit = new Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build();

ApiService api = retrofit.create(ApiService.class);
api.getUser(1).enqueue(new Callback<User>() {
    public void onResponse(Call<User> c, Response<User> r) { /* use r.body() */ }
    public void onFailure(Call<User> c, Throwable t) { /* handle error */ }
});
```

How the three connect: a web service exposes data, JSON is the format that data travels in, and Firebase is a ready-made backend that offers JSON-based storage and APIs so you do not build your own web service.